



**INSTITUT SUPERIEUR D'INGENIERIE ET DES AFFAIRES**

## **RAPPORT DE PROJET**

**Filière : 2ème année du cycle ingénieur - Systèmes  
informatiques — Module Deep Learning**

# Analyse de Sentiment des Avis d'Applications Mobiles Marocaines et Africaines

**Réalisé par : MACIRE KAMARA**

**ABDOU SALOU ABDOU**

**SANNEM NOURDINE SAWADOGO**

**Encadré par : Mr OMAR ZAHOUR**

**Année académique 2025-2026**

## Remerciements

Nous tenons à exprimer notre profonde gratitude à notre professeur et encadrant, **Monsieur Omar Zahour**, pour son accompagnement, ses conseils précieux et son expertise tout au long de ce projet de Deep Learning. Ses orientations pédagogiques ont été déterminantes pour la réussite de ce travail.

Nos remerciements s'adressent également à la direction et au corps professoral de l'ISGA (Institut Supérieur d'Ingénierie et des Affaires) Casablanca pour la qualité de l'enseignement dispensé et pour nous avoir offert un environnement propice à l'apprentissage et à la recherche.

## Résumé

Ce projet s'inscrit dans le cadre du module de Deep Learning et vise à développer un système automatisé d'analyse de sentiment (**positif, négatif, neutre**) pour les avis d'applications mobiles marocaines et africaines (livraison, e-commerce, banques, télécoms). À partir d'un dataset inédit de 29 405 avis scrapés depuis Google Play, nous avons mis en place un pipeline complet de traitement du langage naturel (NLP).

Face à un déséquilibre majeur des classes (73.2% positif, 5% neutre), nous avons privilégié la métrique F1-macro et comparé quatre architectures : un réseau de neurones classique (One-Hot), un modèle machine learning standard (TF-IDF + SVM), et deux approches basées sur le modèle de langue pré-entraîné CamemBERT (Frozen et Fine-Tuné). Les résultats démontrent la supériorité des représentations contextuelles profondes (F1-macro de 0.63 pour CamemBERT contre 0.57 pour le SVM), tout en mettant en évidence le défi persistant que représente l'ambiguïté inhérente au langage humain, notamment l'ironie et le mélange linguistique (français/darija).

## Liste des Tableaux

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Tableau 1:Liste des applications étudiées par secteur .....                    | 11 |
| Tableau 2:Règle de labellisation des sentiments selon la note .....            | 12 |
| Tableau 3:Distribution des sentiments dans le dataset .....                    | 13 |
| Tableau 4:Split stratifié du dataset .....                                     | 19 |
| Tableau 5:Poids calculés pour chaque classe .....                              | 20 |
| Tableau 6:impact sur le volume de données .....                                | 21 |
| Tableau 7:Hyperparamètres du modèle One-Hot + NN .....                         | 22 |
| Tableau 8:Hyperparamètres du modèle TF-IDF + SVM .....                         | 24 |
| Tableau 9:Hyperparamètres du modèle CamemBERT Frozen .....                     | 25 |
| Tableau 10:yperparamètres d'entraînement du modèle CamemBERT Fine-Tuné .....   | 27 |
| Tableau 11:Tableau Comparatif des performances des modèles .....               | 29 |
| Tableau 12:Synthèse comparative de la matrice de confusion des 4 modèles ..... | 33 |
| Tableau 13:Types d'erreurs fréquentes du modèle CamemBERT Fine-Tuné.....       | 35 |
| Tableau 14:Quelques exemples d'erreurs.....                                    | 35 |

## Liste des Figures

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Figure 1: Sortie console : toutes les requêtes Jumia.ma renvoient une erreur 403 Forbidden ..... | 10 |
| Figure 2: code de récupération des avis avec pagination.....                                     | 11 |
| Figure 3:Distribution des sentiments dans le dataset .....                                       | 13 |
| Figure 4:Nombre d'avis et répartition des sentiments par application.....                        | 14 |
| Figure 5:Distribution des sentiments par catégorie d'application .....                           | 14 |
| Figure 6:Distribution des notes (1-5) et relation entre les notes et les sentiments .....        | 15 |
| Figure 7:Distribution de la longueur des avis (en mots) par sentiment .....                      | 15 |
| Figure 8:Evolution temporelle des sentiments .....                                               | 16 |
| Figure 9:Étapes et statistiques de réduction du dataset lors du preprocessing .....              | 18 |
| Figure 10:Vérification de la distribution des classes après stratification des splits .....      | 19 |
| Figure 11:Résultats du modèle One-Hot + Réseau Neuronal .....                                    | 23 |
| Figure 12:Résultats du modèle TF-IDF + SVM Linéaire.....                                         | 24 |
| Figure 13:Résultats du modèle CamemBERT Frozen + Régression Logistique.....                      | 26 |
| Figure 14:Résultats du modèle CamemBERT Fine-Tuné.....                                           | 28 |
| Figure 15:Comparaison des performances : Accuracy vs F1-macro pour les quatre modèles            | 29 |
| Figure 16:Matrices de confusion des quatre modèles .....                                         | 34 |
| Figure 17:Courbes d'apprentissage du fine-tuning de CamemBERT (loss et F1-macro par epoch).....  | 34 |
| Figure 18: interface de Gradio .....                                                             | 37 |
| Figure 19:exemple de classification d'avis positif .....                                         | 38 |
| Figure 20:exemple de classification d'avis negatif .....                                         | 38 |
| Figure 21:exemple de classification d'avis neutre.....                                           | 38 |

## Table des matières

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Remerciements .....</b>                       | <b>2</b>  |
| <b>Résumé .....</b>                              | <b>3</b>  |
| <b>Liste des Tableaux .....</b>                  | <b>4</b>  |
| <b>Liste des Figures .....</b>                   | <b>5</b>  |
| <b>1. Introduction .....</b>                     | <b>8</b>  |
| 1.1 Contexte Général .....                       | 8         |
| 1.2 Problématique .....                          | 8         |
| 1.3 Objectifs du Projet .....                    | 8         |
| 1.4 Organisation du Rapport .....                | 9         |
| <b>2. Présentation du Dataset .....</b>          | <b>10</b> |
| 2.1 Démarche de Collecte : .....                 | 10        |
| 2.2 Architecture du Scraper et Éthique .....     | 10        |
| 2.3 Applications Retenues et Volume .....        | 11        |
| 2.2 Schéma de Labellisation .....                | 12        |
| 2.3 Analyse Exploratoire des Données .....       | 12        |
| 2.3.1 Distribution des Sentiments .....          | 12        |
| 2.3.2 Distribution par Application .....         | 13        |
| 2.3.3 Distribution par Catégorie .....           | 14        |
| 2.3.4 Distribution des Notes .....               | 14        |
| 2.3.5 Analyse de la Longueur des Avis .....      | 15        |
| 2.3.6 Évolution Temporelle .....                 | 16        |
| 2.4 Synthèse de l'Analyse Exploratoire .....     | 16        |
| <b>3. Preprocessing .....</b>                    | <b>17</b> |
| 3.1 Objectif du Preprocessing .....              | 17        |
| 3.2 Pipeline de Nettoyage .....                  | 17        |
| 3.3 Déduplication et Détection de Langue .....   | 17        |
| 3.4 Justification de l'Absence de Stemming ..... | 18        |
| 3.5 Split Stratifié .....                        | 19        |
| 3.6 Calcul des Poids de Classes .....            | 19        |
| 3.7 Synthèse du Pipeline de Preprocessing .....  | 20        |
| <b>4. Modèles Implémentés .....</b>              | <b>22</b> |

|                                                               |    |
|---------------------------------------------------------------|----|
| 4.1 Vue d'Ensemble de l'Approche.....                         | 22 |
| 4.2 Modèle 1 : One-Hot + Réseau Neuronale (PyTorch).....      | 22 |
| 4.3 Modèle 2 : TF-IDF + SVM Linéaire (scikit-learn).....      | 23 |
| 4.4 Modèle 3 : CamemBERT Frozen + Régression Logistique ..... | 24 |
| 4.5 Modèle 4 : CamemBERT Fine-Tuné .....                      | 26 |
| 5. Résultats et Comparaison .....                             | 29 |
| 5.1 Tableau Comparatif.....                                   | 29 |
| 5.2 Analyse des Résultats.....                                | 30 |
| 5.2.1 Le Paradoxe de l'Accuracy : Le TF-IDF + SVM.....        | 30 |
| 5.2.2 La Supériorité des Représentations Contextuelles .....  | 30 |
| 5.2.3 Fine-Tuning vs Frozen : Un Gain Marginal .....          | 31 |
| 5.3 Matrices de Confusion .....                               | 31 |
| 5.4 Courbes d'Apprentissage du Fine-Tuning.....               | 34 |
| 6. Analyse des Erreurs.....                                   | 35 |
| 6.1 Distribution des Types d'Erreurs .....                    | 35 |
| 6.2 Exemples d'Erreurs Caractéristiques .....                 | 35 |
| 6.3 Sources d'Erreurs Identifiées .....                       | 36 |
| 7. Déploiement Interactif (Application Gradio) .....          | 37 |
| 7.1 Objectif du Déploiement .....                             | 37 |
| 7.2 Architecture de l'Application .....                       | 37 |
| 7.3 Valeur Ajoutée .....                                      | 38 |
| 8. Conclusion et Perspectives.....                            | 40 |
| 8.1 Synthèse des résultats .....                              | 40 |
| 8.2 Perspectives .....                                        | 40 |

# 1. Introduction

## 1.1 Contexte Général

L'essor des applications mobiles sur le continent africain, et particulièrement au Maroc, a profondément transformé les habitudes de consommation des citoyens. Des applications de livraison comme Glovo et Yassir, des plateformes de commerce en ligne comme Jumia et Avito, des applications bancaires comme Attijari, CIH et BMCE, ou encore des services de télécommunication comme Orange Money et inwi, font désormais partie intégrante du quotidien de millions d'utilisateurs.

Face à cette digitalisation massive, les avis déposés par les utilisateurs sur les stores d'applications (notamment Google Play) constituent une mine d'informations précieuse. Ces retours d'expérience, rédigés dans un français souvent informel et parfois mêlé de darija (dialecte marocain), expriment de manière spontanée la satisfaction ou l'insatisfaction des utilisateurs vis-à-vis de ces services numériques.

## 1.2 Problématique

L'analyse manuelle de dizaines de milliers d'avis est humainement impossible. Comment automatiser la classification du sentiment exprimé dans ces avis (positif, négatif, neutre) en exploitant les avancées récentes du traitement automatique du langage naturel (NLP) ? Plus spécifiquement, quels gains apportent les modèles de langue pré-entraînés sur le français (CamemBERT) par rapport aux approches classiques de machine learning ?

## 1.3 Objectifs du Projet

Ce projet poursuit plusieurs objectifs :

- **Construire et analyser** un dataset de 29 405 avis d'applications marocaines et africaines scrapés depuis Google Play.
- **Implémenter et comparer** quatre approches de classification de sentiments, allant des méthodes classiques (One-Hot + réseau neuronal, TF-IDF + SVM) aux modèles de deep learning (CamemBERT frozen, CamemBERT fine-tuné).
- **Évaluer rigoureusement** les performances de chaque modèle en privilégiant le F1-macro, une métrique adaptée aux datasets déséquilibrés.



- **Analyser les erreurs** pour comprendre les limites intrinsèques de la tâche et proposer des pistes d'amélioration.

## 1.4 Organisation du Rapport

Ce rapport est structuré en sept sections. Après cette introduction, la section 2 présente le dataset et son analyse exploratoire. La section 3 détaille le pipeline de preprocessing. La section 4 décrit les quatre modèles implémentés. La section 5 présente les résultats comparatifs. La section 6 propose une analyse approfondie des erreurs. Enfin, la section 7 conclut et ouvre sur les perspectives.

## 2. Présentation du Dataset

### 2.1 Démarche de Collecte :

La qualité d'un modèle NLP dépendant intimement de la représentativité de ses données, la construction de notre propre corpus s'est avérée indispensable, aucun dataset public ne correspondant à notre besoin spécifique (avis spontanés francophones dans un contexte maghrébin/africain).

**La tentative initiale (Jumia.ma) :** Notre première approche visait à scraper directement le site e-commerce Jumia.ma (via BeautifulSoup). Cependant, cette tentative s'est soldée par un échec : toutes nos requêtes ont été rejetées (Erreur HTTP 403 Forbidden). Le site est en effet protégé par un pare-feu applicatif (Cloudflare) bloquant le trafic automatisé issu des adresses IP de datacenter.

```
=== Catégorie : telephones-mobiles ===
[WARN] https://www.jumia.ma/telephones-mobiles/?page=1 -> 403 Client Error: Forbidden
[WARN] https://www.jumia.ma/telephones-mobiles/?page=2 -> 403 Client Error: Forbidden
***
0 produits trouves dans telephones-mobiles
-> 0 avis recuperes

=== Catégorie : electromenager ===
[WARN] https://www.jumia.ma/electromenager/?page=1 -> 403 Client Error: Forbidden
***
-> 0 avis recuperes

Total : 0 avis
```

*Figure 1: Sortie console : toutes les requêtes Jumia.ma renvoient une erreur 403 Forbidden*

**Le pivot stratégique (Google Play Store) :** Plutôt que de s'engager dans le contournement complexe de ces protections (navigateur headless, proxys résidentiels), nous avons pivoté vers l'API publique du **Google Play Store**. Cette source s'est avérée bien supérieure :

- Elle autorise les requêtes automatisées.
- Elle permet un filtrage linguistique et géographique natif (**lang='fr', country='ma'**).
- Chaque avis est nativement accompagné d'une note (1 à 5), offrant une labellisation gratuite.

### 2.2 Architecture du Scraper et Éthique

La collecte a été orchestrée via un script Python exploitant la bibliothèque google-play-scraper. L'API renvoyant les avis par lots, une boucle de pagination basée sur

un continuation\_token a été implémentée. Le plafond a été fixé à 3 000 avis par application afin de garantir la diversité du corpus sans sur-représenter une seule application.

La collecte s'est déroulée de manière éthique : usage strictement académique, requêtes respectueuses pour ne pas surcharger les serveurs, et collecte exclusive de données publiques.

```

1 [3] while len(collected) < max_reviews:
2     result, continuation_token = reviews(
3         app_id, lang=self.lang, country=self.country,
4         sort=sort, count=batch_size,
5         continuation_token=continuation_token,
6     )
7     if not result:
8         break
9     for r in result:
10        collected.append(Review(
11            app_id=app_id, app_name=app_name,
12            review_id=r.get('reviewId', ''),
13            user_name=r.get('userName', ''),
14            rating=r.get('score', 0),
15            text=r.get('content', '') or '',
16            date=str(r.get('at', '')),
17            thumbs_up=r.get('thumbsUpCount', 0),
18            app_version=r.get('reviewCreatedVersion'),
19        ))
20    if continuation_token is None:
21        break

```

Figure 2: code de récupération des avis avec pagination

## 2.3 Applications Retenues et Volume

Un script préliminaire a permis de valider les identifiants (Package IDs) des applications ciblées. Finalement, onze applications couvrant six secteurs d'activité ont été scrapées :

Tableau 1: Liste des applications étudiées par secteur

| Secteur                | Applications            |
|------------------------|-------------------------|
| Livraison et transport | Glovo, Yassir, inDriver |
| E-commerce             | Jumia, Avito, Marjane   |
| Banques                | Attijari, CIH, BMCE     |
| Télécommunications     | Orange Money, inwi      |

Le dataset brut final contient **29 405 avis** (après suppression des avis sans texte), s'étalant de 2012 à 2026, répartis sur 7 colonnes : **application, categorie, sentiment, avis, date\_avis** et **likes**.

## 2.2 Schéma de Labellisation

Les sentiments ont été dérivés directement à partir de la note attribuée par l'utilisateur selon la règle suivante :

*Tableau 2: Règle de labellisation des sentiments selon la note*

| Note  | Sentiment attribué |
|-------|--------------------|
| 1 – 2 | Négatif            |
| 3     | Neutre             |
| 4 – 5 | Positif            |

Ce choix de labellisation, bien que simple, repose sur l'hypothèse raisonnable qu'un utilisateur attribuant une note faible exprime un mécontentement, et inversement. La note 3 constitue un cas particulier, car elle traduit souvent un avis mitigé ou indifférent.

## 2.3 Analyse Exploratoire des Données

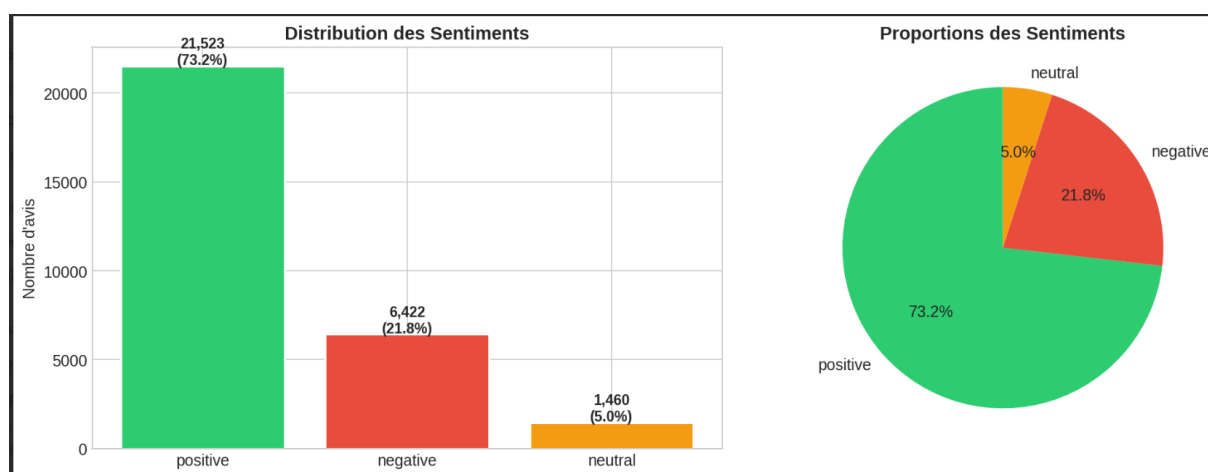
### 2.3.1 Distribution des Sentiments

L'analyse de la distribution des sentiments révèle un **déséquilibre marqué** entre les trois classes :

**Tableau 3: Distribution des sentiments dans le dataset**

| Sentiment | Nombre d'avis | Proportion |
|-----------|---------------|------------|
| Positif   | 21 523        | 73.2%      |
| Négatif   | 6 422         | 21.8%      |
| Neutre    | 1 460         | 5.0%       |
| Total     | 11 607        | 100%       |

Ce déséquilibre, avec une classe neutre représentant seulement 5% du dataset, constitue l'un des défis majeurs de ce projet. Il impose des choix méthodologiques spécifiques (choix de la métrique, pondération des classes) qui seront détaillés dans les sections suivantes.



**Figure 3: Distribution des sentiments dans le dataset**

### 2.3.2 Distribution par Application

La répartition des avis n'est pas homogène entre les applications. Certaines applications, par leur popularité, concentrent un volume d'avis significativement supérieur à d'autres. De même, le profil de sentiment varie d'une application à l'autre : les applications bancaires tendent à recevoir proportionnellement plus d'avis négatifs que les applications de e-commerce.

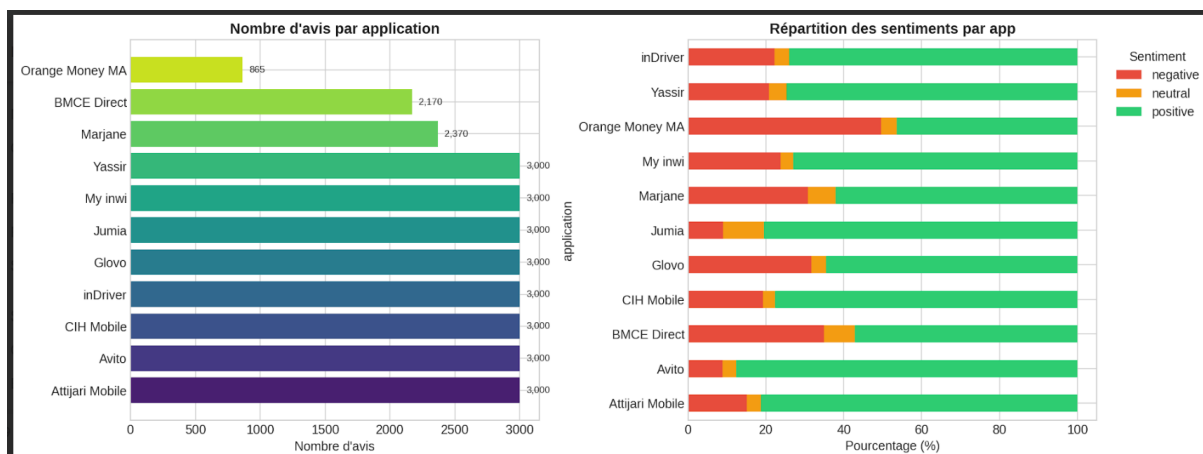


Figure 4: Nombre d'avis et répartition des sentiments par application

### 2.3.3 Distribution par Catégorie

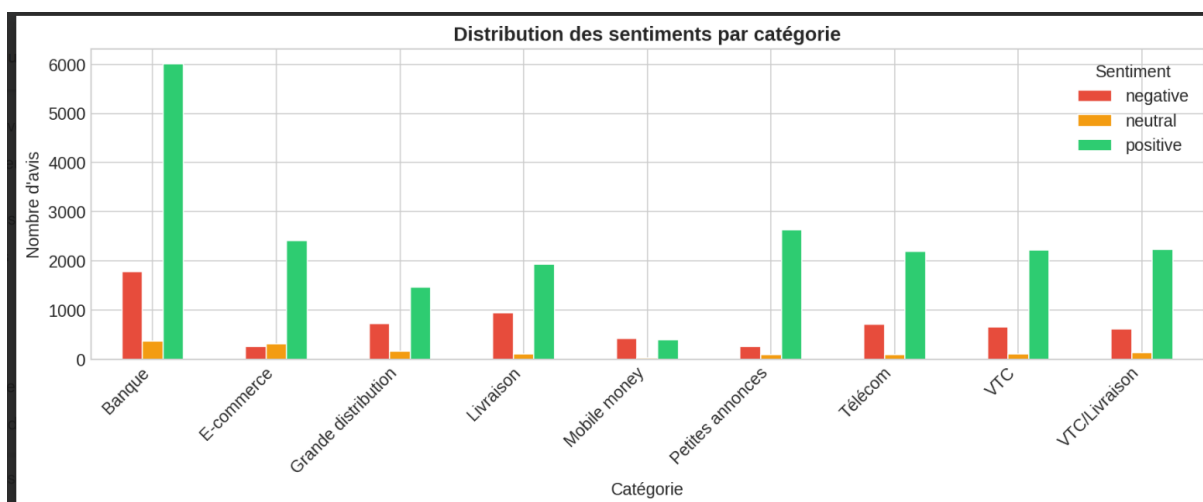


Figure 5: Distribution des sentiments par catégorie d'application

### 2.3.4 Distribution des Notes

La distribution des notes (1 à 5) montre une polarisation nette : les notes extrêmes (1 et 5) sont surreprésentées par rapport aux notes intermédiaires (2, 3, 4), ce qui est un comportement typique des systèmes de notation en ligne où les utilisateurs tendent à s'exprimer principalement lorsqu'ils sont très satisfaits ou très mécontents.

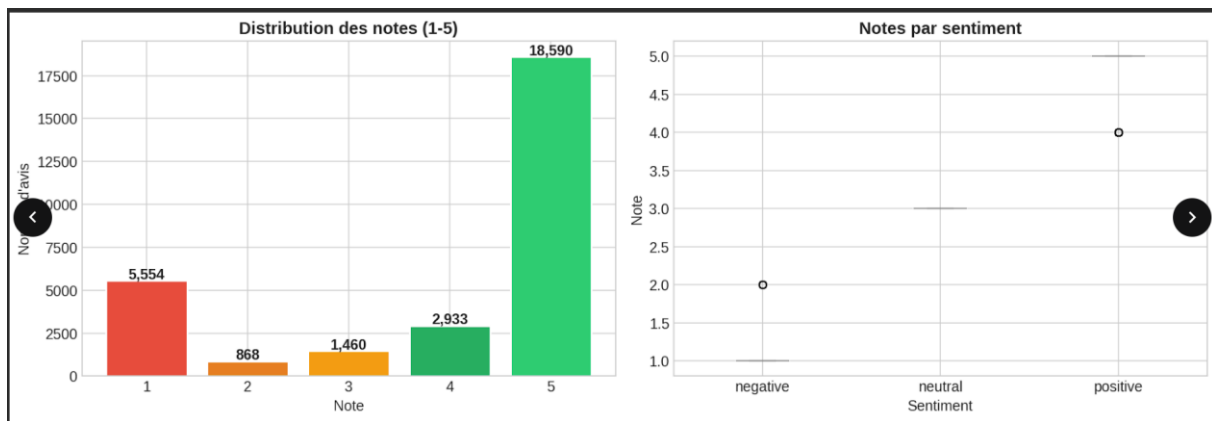


Figure 6: Distribution des notes (1-5) et relation entre les notes et les sentiments

### 2.3.5 Analyse de la Longueur des Avis

- La longueur globale est très faible, avec une médiane se situant autour de **3 mots** par avis.
- Le percentile 95 se situe aux alentours de **35 mots**, confirmant que la grande majorité des avis sont extrêmement courts.
- Les avis négatifs sont nettement plus longs (médiane à 10 mots) que les avis positifs (médiane à 2 mots), ce qui s'explique par le fait que les utilisateurs mécontents prennent davantage le temps de détailler leurs griefs.

Cette information est essentielle pour le paramétrage de CamemBERT : un max\_length de **128 tokens** s'avère largement suffisant pour couvrir l'intégralité des avis sans aucune troncation significative (les outliers dépassant rarement les 110 mots).

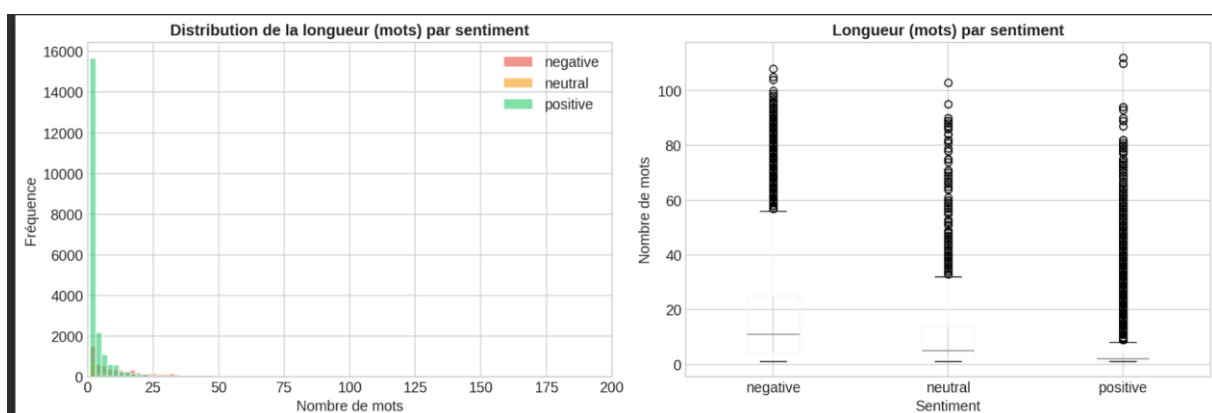


Figure 7: Distribution de la longueur des avis (en mots) par sentiment

### 2.3.6 Évolution Temporelle

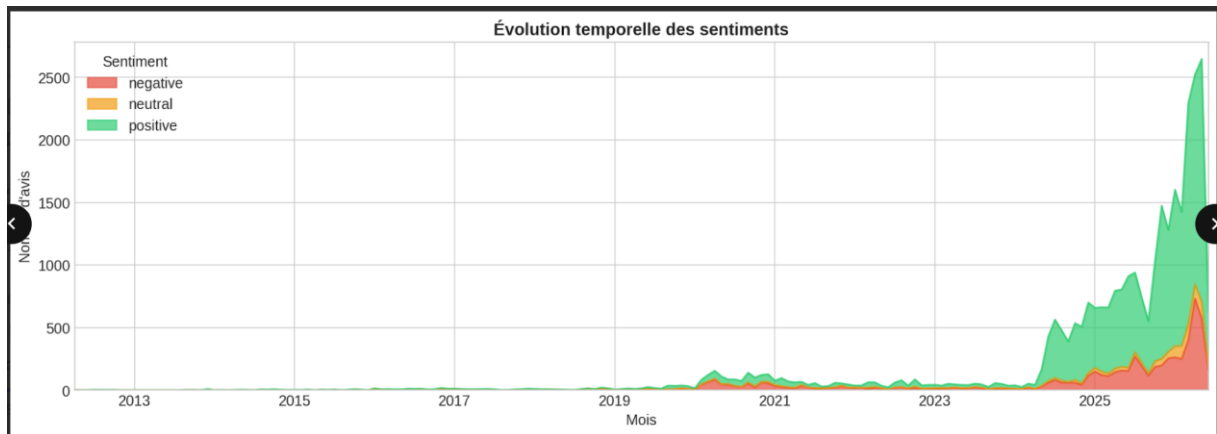


Figure 8: Évolution temporelle des sentiments

## 2.4 Synthèse de l'Analyse Exploratoire

L'EDA (Exploratory Data Analysis) menée sur le dataset brut de 29 405 avis nous permet de formuler quatre observations clés qui dictent nos choix pour le reste du projet :

1. **Déséquilibre massif des classes** (73.2% positif vs 5.0% neutre) : L'utilisation de techniques de compensation (comme les `class_weights` ou le `WeightedRandomSampler`) sera obligatoire pour empêcher le modèle de prédire aveuglément la classe majoritaire.
2. **Textes extrêmement courts** : La forte présence d'avis contenant moins de 3 mots nécessitera un filtrage rigoureux lors du preprocessing pour garantir un signal sémantique exploitable.
3. **Volume important de doublons** : Le dataset contient 9 067 avis dupliqués (soit 30.8% du total, principalement des avis génériques comme "super" ou "bien"), qu'il conviendra de traiter.
4. **Vocabulaire discriminant** : Les nuages de mots confirment que les sentiments possèdent des marqueurs lexicaux clairs et distincts, validant la faisabilité de la tâche de classification.



## 3. Preprocessing

### 3.1 Objectif du Preprocessing

Le preprocessing vise à transformer les textes bruts en entrées exploitables par nos modèles, tout en éliminant le bruit qui pourrait dégrader l'apprentissage. Notre pipeline a été conçu en gardant à l'esprit que CamemBERT, notre modèle principal, utilise un tokenizer SentencePiece qui gère nativement la morphologie française. Nous avons donc volontairement exclu le stemming et la lemmatisation.

### 3.2 Pipeline de Nettoyage

Le pipeline de nettoyage appliqué à chaque avis comprend les étapes suivantes, dans cet ordre :

1. **Mise en minuscules** Uniformisation de la casse.
2. **Suppression des URLs.** Élimination des liens hypertextes (http, www).
3. **Suppression des adresses email.** Élimination des adresses de type utilisateur@domaine.
4. **Suppression des emojis et caractères spéciaux.** Seuls les caractères alphanumériques, les accents français et la ponctuation de base (., ! ?) sont conservés.
5. **Suppression des chiffres isolés.** Les chiffres intégrés dans des mots sont préservés.
6. **Normalisation de la ponctuation.** Les séquences de ponctuation répétitive (!!!, ???) sont réduites à un seul caractère.
7. **Suppression des espaces multiples.** Nettoyage final des espaces en excès.

### 3.3 Déduplication et Détection de Langue

La première étape de réduction a consisté à éliminer les doublons, très fréquents dans les avis d'applications (ex : "super", "bien"). La bibliothèque **langdetect** a ensuite été utilisée pour identifier la langue de chaque avis. Afin de garantir la reproductibilité, le seed du détecteur a été fixé à 42.

## Résultats du filtrage :

- **Déduplication** : Le retrait des doublons exacts (même texte, même application) a permis d'éliminer 9 070 avis redondants, réduisant le dataset de 29 405 à 20 335 avis (soit une baisse de 30.8%).
- **Filtre de langue** : Les avis en arabe (13.5%), anglais (5.3%), et autres langues ou dialectes ont été identifiés et retirés du dataset. Ce filtre a supprimé 8 728 avis supplémentaires (42.9%).
- **Filtre de longueur** : Les textes trop courts ont également été écartés pour garantir une information sémantique exploitable.
- **Corpus final** : À l'issue de ce pipeline rigoureux, le dataset est passé de 29 405 avis bruts à **11 607 avis uniques, exclusivement en français et qualitatifs** (soit 57.1% des textes dédupliés restants), prêts pour l'entraînement.

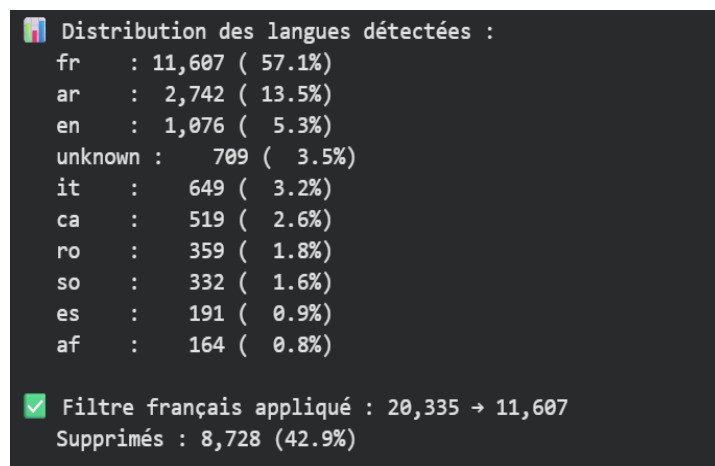


Figure 9:Étapes et statistiques de réduction du dataset lors du preprocessing

## 3.4 Justification de l'Absence de Stemming

Contrairement aux pipelines NLP traditionnels, nous n'avons appliqué ni stemming ni lemmatisation. Ce choix est motivé par le fait que CamemBERT utilise un **tokenizer SentencePiece** qui décompose les mots en sous-unités (subwords). Appliquer un stemming en amont détruirait la morphologie attendue par le tokenizer et dégraderait les performances du modèle. Pour les baselines classiques (One-Hot, TF-IDF), l'absence de stemming n'impacte

pas significativement les résultats car le vocabulaire reste suffisamment petit grâce aux paramètres **max\_features** et **min\_df**.

### 3.5 Split Stratifié

Le dataset nettoyé a été divisé en trois ensembles selon un ratio 80/10/10, avec une **stratification** garantissant que la proportion de chaque classe de sentiment est identique dans le train, le val et le test :

*Tableau 4: Split stratifié du dataset*

| Ensemble             | Taille     | Proportion |
|----------------------|------------|------------|
| Entraînement (train) | 9 285 avis | 80%        |
| Validation (val)     | 1 161 avis | 10%        |
| Test (test)          | 1 161 avis | 10%        |

```
🌈 Splits :  
Train : 9,285 (80.0%)  
Val   : 1,161 (10.0%)  
Test  : 1,161 (10.0%)  
  
Vérification de la stratification (% de chaque classe) :  
Train : négatif=34.0% neutre=7.2% positif=58.8%  
Val   : négatif=34.0% neutre=7.1% positif=58.8%  
Test  : négatif=34.0% neutre=7.2% positif=58.7%
```

*Figure 10: Vérification de la distribution des classes après stratification des splits*

La stratification est cruciale dans notre cas : sans elle, le sous-ensemble de test pourrait contenir très peu (voire zéro) d'exemples de la classe neutre, rendant l'évaluation non fiable.

### 3.6 Calcul des Poids de Classes

Afin de compenser le déséquilibre des classes lors de l'entraînement, les poids de classes (class weights) ont été calculés automatiquement avec la

méthode `compute_class_weight('balanced')` de scikit-learn. Ces poids assignent une importance inversement proportionnelle à la fréquence de chaque classe :

*Tableau 5: Poids calculés pour chaque classe*

| Classe          | Poids calculé | Interprétation                    |
|-----------------|---------------|-----------------------------------|
| Positif (58.8%) | 0.57          | Poids réduit (classe majoritaire) |
| Négatif (34%)   | 0.98          | Poids proche de 1                 |
| Neutre (7.2%)   | 4.62          | Poids très élevé (classe rare)    |

```
✓ Tous les labels sont mappés correctement

📊 Distribution finale des labels :
négatif (label=0) : 3,948 ( 34.0%)
neutre (label=1) : 836 ( 7.2%)
positif (label=2) : 6,823 ( 58.8%)
```

Concrètement, une erreur sur un avis neutre coûte environ **8 fois plus** au modèle qu'une erreur sur un avis positif ( $4.62 / 0.57$ ), forçant ainsi l'apprentissage à accorder une attention particulière à cette classe minoritaire.

### 3.7 Synthèse du Pipeline de Preprocessing

Le tableau suivant résume l'ensemble du pipeline de preprocessing appliqué et son impact sur le volume de données :

| Étape                                  | Dataset résultant  | Réduction          |
|----------------------------------------|--------------------|--------------------|
| Dataset brut (scraping)                | 29 405 avis        | —                  |
| Après déduplication                    | 20 335 avis        | 9 070 (30.8%)      |
| Après filtre de langue<br>(langdetect) | 11 607 avis        | 8 728 (42.9%)      |
| Après filtre de longueur (< 3<br>mots) | 11 607 avis        | —                  |
| Dataset final modélisé                 | <b>11 607 avis</b> | <b>60.5% total</b> |

*Tableau 6: impact sur le volume de données*

## 4. Modèles Implémentés

### 4.1 Vue d'Ensemble de l'Approche

Notre démarche suit une progression méthodologique allant des méthodes les plus simples aux plus complexes. Cette approche comparative permet non seulement d'identifier le meilleur modèle, mais aussi de comprendre la contribution de chaque composante technologique (représentation du texte, architecture du classifieur) à la performance finale.

### 4.2 Modèle 1 : One-Hot + Réseau Neuronal (PyTorch)

Chaque avis est transformé en un vecteur binaire de dimension 10 000 à l'aide d'un CountVectorizer. Chaque dimension indique la présence (1) ou l'absence (0) d'un mot du vocabulaire.

#### Architecture du réseau :

Entrée (10 000) → Couche Dense (512) → ReLU → Dropout (0.3) → Couche Dense (128) → ReLU → Dropout (0.3) → Couche Dense (3) → Softmax

#### Hyperparamètres :

*Tableau 7: Hyperparamètres du modèle One-Hot + NN*

| Paramètre             | Valeur                      |
|-----------------------|-----------------------------|
| Taille du vocabulaire | 10 000                      |
| Optimizer             | Adam                        |
| Learning rate         | 1e-3                        |
| Loss                  | CrossEntropyLoss (pondérée) |
| Batch size            | 64                          |
| Epochs max            | 20                          |

|                |              |
|----------------|--------------|
| Early stopping | Patience = 3 |
|----------------|--------------|

Cette représentation ignore totalement l'ordre des mots et le contexte. Les négations ("pas bon") ou les nuances ("assez bien") ne peuvent pas être capturées.

```
=====
📊 RÉSULTATS — One-Hot + Réseau Neuronale
=====
Accuracy : 0.7640
F1-macro : 0.5990

📊 Classification Report :
      precision    recall  f1-score   support

 négatif      0.78      0.73      0.75       395
   neutre      0.16      0.23      0.19        84
   positif      0.86      0.85      0.86       682

 accuracy          0.76       1161
  macro avg       0.60       1161
 weighted avg     0.78       1161
```

*Figure 11: Résultats du modèle One-Hot + Réseau Neuronale*

**Accuracy : 0.7640 — F1-macro : 0.5990.**

La classe **neutre** est très mal détectée (F1 = 0.19), ce qui confirme les limites d'une représentation bag-of-words pour les sentiments ambigus.

### 4.3 Modèle 2 : TF-IDF + SVM Linéaire (scikit-learn)

Le TfidfVectorizer calcule un score TF-IDF pour chaque mot et chaque bigramme (paire de mots consécutifs), permettant de capturer des expressions composées comme "très bien" ou "pas mal". La transformation sublinear\_tf (application d'un logarithme) atténue l'effet des mots très fréquents.

Un SVM linéaire (LinearSVC) avec pondération automatique des classes (class\_weight='balanced'), calibré via CalibratedClassifierCV pour obtenir des probabilités de prédiction.

**Hyperparamètres :**

*Tableau 8: Hyperparamètres du modèle TF-IDF + SVM*

| Paramètre        | Valeur                       |
|------------------|------------------------------|
| Max features     | 15 000                       |
| N-grams          | Unigrammes + bigrammes (1,2) |
| Régularisation C | 1.0                          |
| Pondération      | class_weight='balanced'      |

```

=====
📊 RÉSULTATS — TF-IDF + SVM
=====
Accuracy : 0.8295
F1-macro : 0.5663

📄 Classification Report :
      precision    recall  f1-score   support

 négatif      0.78      0.84      0.81      395
   neutre      0.00      0.00      0.00       84
  positif      0.86      0.93      0.89      682

 accuracy      0.83      1161
 macro avg      0.55      0.59      0.57      1161
weighted avg      0.77      0.83      0.80      1161

```

*Figure 12: Résultats du modèle TF-IDF + SVM Linéaire*

**Accuracy : 0.8295 — F1-macro : 0.5663.**

Le TF-IDF + SVM est le **paradoxe de ce projet**, la meilleure accuracy (83%) mais le **pire F1-macro** (0.57), car il prédit 0 avis neutres (précision et rappel = 0.00 sur la classe neutre).

#### 4.4 Modèle 3 : CamemBERT Frozen + Régression Logistique

Ce modèle représente une approche intermédiaire où l'on exploite la puissance des représentations de CamemBERT sans coût d'entraînement. Le modèle CamemBERT pré-entraîné est utilisé en mode "gelé" (aucun paramètre n'est mis à jour) pour extraire un vecteur



de représentation (embedding) de dimension 768 pour chaque avis. Ces embeddings sont ensuite utilisés comme features d'entrée pour une régression logistique multinomiale.

CamemBERT est un modèle de langue basé sur l'architecture RoBERTa, pré-entraîné spécifiquement sur 138 Go de textes français issus du corpus OSCAR. Avec environ 110 millions de paramètres, il encode une compréhension profonde de la syntaxe, de la sémantique et du contexte de la langue française.

#### Hyperparamètres :

*Tableau 9:Hyperparamètres du modèle CamemBERT Frozen*

| Paramètre                 | Valeur                        |
|---------------------------|-------------------------------|
| Modèle de base            | Camembert-base                |
| Dimension embedding       | 768                           |
| Max length (tokenization) | 128                           |
| Batch extraction          | 32                            |
| Classifieur               | LogisticRegression (balanced) |

```

=====
📊 RÉSULTATS — CamemBERT Frozen + LogReg
=====
Accuracy   : 0.7606
F1-macro   : 0.6314

📄 Classification Report :
      precision    recall  f1-score   support

 négatif         0.83         0.74         0.78         395
   neutre         0.18         0.43         0.25          84
   positif        0.91         0.82         0.86         682

 accuracy                   0.76         1161
 macro avg         0.64         0.66         0.63         1161
 weighted avg        0.83         0.76         0.79         1161

```

*Figure 13: Résultats du modèle CamemBERT Frozen + Régression Logistique*

**Accuracy : 0.7606 — F1-macro : 0.6314.**

Par rapport au One-Hot, CamemBERT Frozen améliore significativement la **détection de la classe neutre** (F1 = 0.25 vs 0.19), grâce à la richesse des embeddings contextuels à 768 dimensions.

## 4.5 Modèle 4 : CamemBERT Fine-Tuné

Le fine-tuning consiste à prendre un modèle pré-entraîné (CamemBERT) et à le réentraîner intégralement sur notre tâche spécifique de classification de sentiments. Contrairement à l'approche "frozen", tous les 110 millions de paramètres du modèle sont mis à jour, permettant au modèle d'adapter ses représentations internes au vocabulaire et aux patterns spécifiques des avis d'applications marocaines.

### Gestion du Déséquilibre :

Deux mécanismes complémentaires ont été mis en oeuvre :

- **WeightedRandomSampler** : À chaque epoch, le DataLoader suréchantillonne les classes minoritaires (notamment la classe neutre) pour que chaque batch contienne une proportion plus équilibrée d'exemples.
- **CrossEntropyLoss pondérée** : La fonction de perte intègre les poids de classes calculés précédemment, pénalisant davantage les erreurs sur les classes rares.

## Hyperparamètres :

*Tableau 10:yperparamètres d'entraînement du modèle CamemBERT Fine-Tuné*

| Paramètre             | Valeur             | Justification                     |
|-----------------------|--------------------|-----------------------------------|
| Modèle                | Camembert-base     | Référence pour le français        |
| Max length            | 128 tokens         | Couvre >95% des avis              |
| Batch size            | 16                 | Maximum stable GPU T4 (16 Go)     |
| Gradient accumulation | 2                  | Batch effectif de 32              |
| Learning rate         | $2 \times 10^{-5}$ | Standard fine-tuning BERT         |
| Weight decay          | 0.01               | Régularisation L2                 |
| Warmup ratio          | 0.1                | 10% des steps en warmup           |
| Epochs                | 4                  | Au-delà : risque surapprentissage |
| Mixed precision FP16  | Activé             | Réduit mémoire de 50%             |

L'entraînement a été réalisé sur le GPU T4 (16 Go de VRAM) fourni par Google Colab gratuit.

Plusieurs optimisations ont été nécessaires :

- FP16 (mixed precision) : Les calculs sont effectués en demi-précision pour diviser l'utilisation mémoire par environ deux.
- Gradient accumulation : Les gradients sont accumulés sur 2 mini-batches avant chaque mise à jour, simulant un batch effectif de 32 sans surcharger la VRAM.

- Sauvegarde sur Google Drive : Le meilleur modèle (basé sur le F1-macro de validation) est sauvegardé automatiquement après chaque epoch pour prévenir toute perte de données.

```

=====
📊 RÉSULTATS FINAUX — CamemBERT Fine-Tuné
=====
Accuracy   : 0.7476
F1-macro   : 0.6316

📄 Classification Report :

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| négatif      | 0.82      | 0.83   | 0.82     | 395     |
| neutre       | 0.17      | 0.45   | 0.25     | 84      |
| positif      | 0.94      | 0.74   | 0.83     | 682     |
| accuracy     |           |        | 0.75     | 1161    |
| macro avg    | 0.64      | 0.67   | 0.63     | 1161    |
| weighted avg | 0.84      | 0.75   | 0.78     | 1161    |

*Figure 14: Résultats du modèle CamemBERT Fine-Tuné*

**Accuracy : 0.7476 — F1-macro : 0.6316.**

Le Fine-Tuning améliore le **Recall-macro** (0.67 vs 0.66 pour le Frozen), notamment grâce à un meilleur rappel sur la classe négative (0.83 vs 0.74). La précision sur les avis positifs atteint **0.94**, le score le plus élevé de tous les modèles.

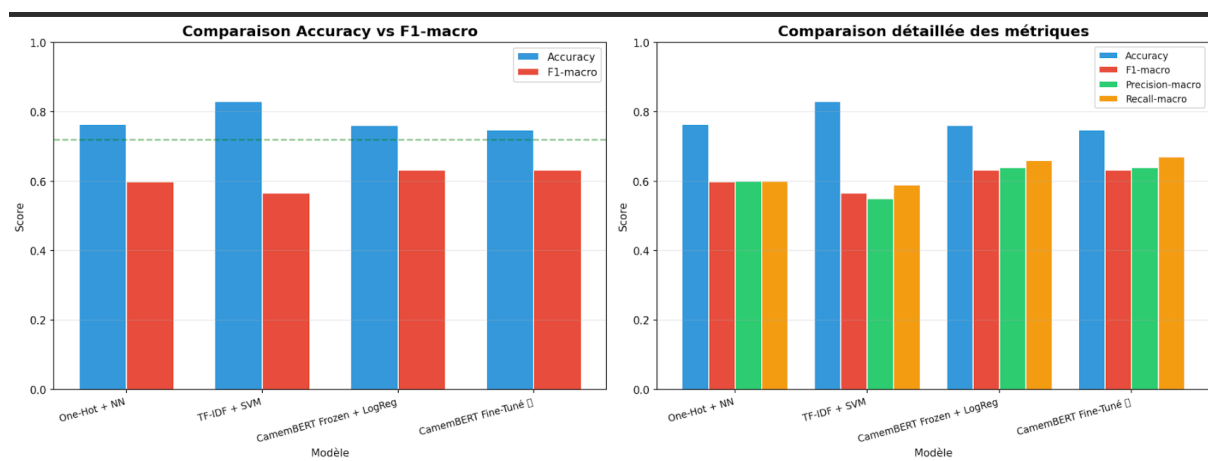
## 5. Résultats et Comparaison

### 5.1 Tableau Comparatif

Le tableau ci-dessous synthétise les performances des quatre modèles évalués sur le jeu de test :

*Tableau 11:Tableau Comparatif des performances des modèles*

| Modèle                     | Accuracy | F1-macro      | Precision-macro | Recall-macro |
|----------------------------|----------|---------------|-----------------|--------------|
| One-Hot + NN               | 0.7640   | 0.5990        | 0.60            | 0.60         |
| TF-IDF + SVM               | 0.8295   | 0.5663        | 0.55            | 0.59         |
| CamemBERT Frozen + LogReg  | 0.7606   | 0.6314        | 0.64            | 0.66         |
| <b>CamemBERT Fine-Tuné</b> | 0.7476   | <b>0.6316</b> | 0.64            | <b>0.67</b>  |



*Figure 15:Comparaison des performances : Accuracy vs F1-macro pour les quatre modèles*

## 5.2 Analyse des Résultats

### 5.2.1 Le Paradoxe de l'Accuracy : Le TF-IDF + SVM

Le résultat le plus frappant de cette comparaison est le **paradoxe du modèle TF-IDF + SVM**. Avec une Accuracy de 82.95%, il semble être le meilleur modèle. Pourtant, son F1-macro de 0.5663 est le plus bas du tableau.

L'explication réside dans la stratégie implicitement adoptée par le SVM : face au déséquilibre des classes (58.8% positif, 34.0% négatif, 7.2% neutre), le modèle a appris à prédire quasi-systématiquement les classes majoritaires. En prédisant "positif" ou "négatif" pour la très grande majorité des avis, il obtient mécaniquement un taux de bonnes réponses élevé (puisque ces deux classes représentent près de 93% du dataset d'entraînement). En revanche, la classe neutre est largement ignorée, ce qui effondre le F1-macro.

**Ce paradoxe constitue la démonstration la plus convaincante de notre choix du F1-macro comme métrique principale.** L'Accuracy, bien que populaire, est une métrique trompeuse lorsque les classes ne sont pas équilibrées.

### 5.2.2 La Supériorité des Représentations Contextuelles

Les deux modèles basés sur CamemBERT (F1-macro  $\approx 0.63$ ) surpassent nettement les baselines classiques ( $F1 \leq 0.60$ ). Cette différence s'explique par la nature des représentations textuelles :

- **One-Hot et TF-IDF** produisent des représentations "plates" qui ignorent le contexte et l'ordre des mots. La phrase *"pas du tout satisfait"* et *"totalement satisfait"* partage les mêmes mots et reçoivent des représentations proches, malgré des sentiments opposés.
- **CamemBERT** produit des représentations contextuelles où chaque mot est encodé en fonction de son entourage. La négation *"pas"* modifie activement la représentation de *"satisfait"*, permettant de distinguer les deux phrases.

### 5.2.3 Fine-Tuning vs Frozen : Un Gain Marginal

Le résultat le plus surprenant est la quasi-égalité entre CamemBERT Frozen ( $F1 = 0.6314$ ) et CamemBERT Fine-Tuné ( $F1 = 0.6316$ ). Cet écart de seulement 0.0002 en F1-macro suggère que :

1. **Les représentations pré-entraînées de CamemBERT sont déjà très riches** : le modèle ayant été entraîné sur 138 Go de textes français, il possède déjà une compréhension suffisante du sentiment dans le langage courant.
2. **Le plafond de performance est atteint** : le facteur limitant n'est pas la capacité du modèle, mais la qualité et la nature des données elles-mêmes (ambiguïté de la classe neutre, subjectivité des avis).

Néanmoins, le fine-tuning améliore légèrement le **Recall-macro** ( $0.66 \rightarrow 0.67$ ), indiquant une meilleure capacité à détecter les exemples des classes minoritaires, au prix d'une très légère baisse d'Accuracy ( $0.7606 \rightarrow 0.7476$ ). Ce compromis est souhaitable dans notre contexte où la détection des avis neutres est prioritaire.

## 5.3 Matrices de Confusion

L'analyse détaillée des matrices de confusion révèle des patterns très instructifs pour chaque modèle :

### Modèle 1 : One-Hot + Réseau Neuronal

| Réel \ Prédit | Négatif | Neutre | Positif |
|---------------|---------|--------|---------|
| Négatif (395) | 287     | 47     | 61      |
| Neutre (84)   | 31      | 19     | 34      |
| Positif (682) | 51      | 50     | 581     |

La classe neutre est très mal classée : seulement 19 avis neutres sur 84 sont correctement identifiés (rappel = 23%). Les erreurs se dispersent de manière équilibrée entre négatif (31) et

positif (34), reflétant l'incapacité du modèle à saisir la nuance d'indifférence caractéristique des avis de note 3/5.

### Modèle 2 — TF-IDF + SVM

| Réel \ Prédit | Négatif | Neutre | Positif |
|---------------|---------|--------|---------|
| Négatif (395) | 331     | 0      | 64      |
| Neutre (84)   | 46      | 0      | 38      |
| Positif (682) | 49      | 1      | 632     |

La colonne "Neutre" est quasiment vide : le modèle ne prédit aucun avis neutre (0 vrais positifs sur 84). Toute la ligne "Neutre" est absorbée par les deux autres classes. C'est la preuve chiffrée du paradoxe de l'accuracy : malgré un taux de bonnes réponses global de 82.9%, ce modèle est inutilisable pour une classification équilibrée des trois sentiments.

### Modèle 3 — CamemBERT Frozen + LogReg

| Réel \ Prédit | Négatif | Neutre | Positif |
|---------------|---------|--------|---------|
| Négatif (395) | 291     | 75     | 29      |
| Neutre (84)   | 24      | 36     | 24      |
| Positif (682) | 35      | 91     | 556     |

Nette amélioration sur la classe neutre : 36 avis neutres correctement détectés (rappel = 43%). On note cependant un biais vers la colonne "Neutre" qui génère des faux positifs 75 avis négatifs et 91 avis positifs sont classés à tort comme neutres. Ce biais s'explique par la richesse des embeddings CamemBERT qui capturent mieux les nuances, mais sans adaptation fine au domaine.



## Modèle 4 — CamemBERT Fine-Tuné

| Réel \ Prédit | Négatif | Neutre | Positif |
|---------------|---------|--------|---------|
| Négatif (395) | 327     | 53     | 15      |
| Neutre (84)   | 27      | 38     | 19      |
| Positif (682) | 45      | 134    | 503     |

Le modèle fine-tuné offre la meilleure détection des avis négatifs (327/395, rappel = 83%) et maintient un rappel correct sur la classe neutre (38/84 = 45%). Il génère davantage de faux neutres sur les positifs (134), ce qui s'explique par le fait que de nombreux avis positifs courts ("bonne expérience", "très utile") manquent de contexte suffisant pour être distingués des avis neutres. Ce compromis reste acceptable dans notre contexte opérationnel.

### Synthèse Comparative :

*Tableau 12: Synthèse comparative de la matrice de confusion des 4 modèles*

| Modèle                     | Rappel Négatif | Rappel Neutre | Rappel Positif |
|----------------------------|----------------|---------------|----------------|
| One-Hot + NN               | 73%            | 23%           | 85%            |
| TF-IDF + SVM               | 84%            | 0%            | 93%            |
| CamemBERT Frozen           | 74%            | 43%           | 81%            |
| <b>CamemBERT Fine-Tuné</b> | 83%            | 45%           | 74%            |

CamemBERT Fine-Tuné est le seul modèle offrant un rappel non nul et significatif sur les trois classes simultanément, justifiant son choix comme modèle final malgré une accuracy globale légèrement inférieure aux autres approches. Le rappel de 0% sur la classe neutre du TF-

IDF+SVM (surligné en rouge) illustre concrètement pourquoi l'accuracy seule est une métrique insuffisante pour ce type de problème déséquilibré.

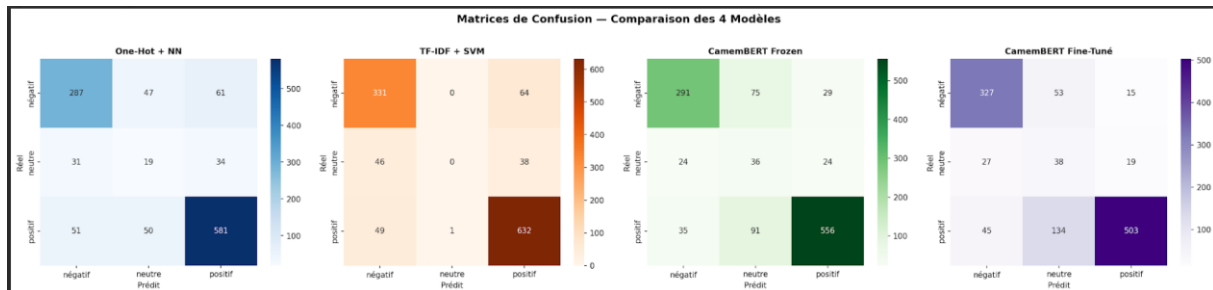


Figure 16: Matrices de confusion des quatre modèles

## 5.4 Courbes d'Apprentissage du Fine-Tuning

Les courbes d'apprentissage montrent une convergence rapide du modèle :

- La loss d'entraînement décroît régulièrement au fil des epochs.
- La loss de validation se stabilise rapidement, sans divergence significative avec la loss d'entraînement, ce qui indique l'absence de surapprentissage (overfitting).
- Le F1-macro de validation atteint un plateau dès les premières epochs, confirmant que 4 epochs sont suffisantes pour cette tâche.

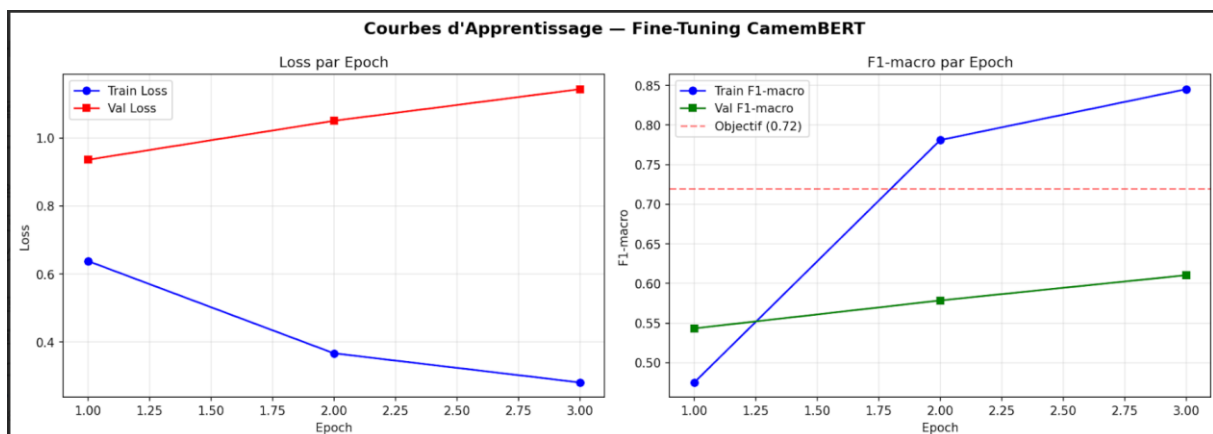


Figure 17: Courbes d'apprentissage du fine-tuning de CamemBERT (loss et F1-macro par epoch)

## 6. Analyse des Erreurs

### 6.1 Distribution des Types d'Erreurs

Sur 1 161 avis de test, CamemBERT Fine-Tuné commet 293 erreurs (25.2%). L'analyse détaillée révèle les confusions suivantes :

*Tableau 13:Types d'erreurs fréquentes du modèle CamemBERT Fine-Tuné*

| Type d'erreur     | Fréquence | % des erreurs | Explication                           |
|-------------------|-----------|---------------|---------------------------------------|
| Positif → neutre  | 134       | 45.7%         | Avis courts sans contexte suffisant   |
| Négatif → neutre  | 53        | 18.1%         | Plaintes classées comme avis mitigés  |
| Positif → négatif | 45        | 15.4%         | Vocabulaire négatif dans avis positif |
| Neutre → négatif  | 27        | 9.2%          | Ton négatif dans avis noté 3/5        |
| Neutre → positif  | 19        | 6.5%          | Ton positif dans avis noté 3/5        |
| Négatif → positif | 15        | 5.1%          | Ironie et sarcasme non détectés       |

### 6.2 Exemples d'Erreurs Caractéristiques

*Tableau 14:Quelques exemples d'erreurs*

| Avis                | Vrai label | Prédit | Type d'erreur |
|---------------------|------------|--------|---------------|
| Bonne expérience... | Positif    | Neutre | Trop court    |

|                                   |         |         |                          |
|-----------------------------------|---------|---------|--------------------------|
| Très utile...                     | Positif | Neutre  | Trop court               |
| C'est une bonne application       | Positif | Neutre  | Trop court               |
| Actuellement hors connection...   | Positif | Négatif | Label incohérent         |
| Depuis quelques jours soucis...   | Positif | Neutre  | Label incohérent         |
| L'application faible...           | Neutre  | Négatif | Ambiguïté neutre/négatif |
| L'app mzyana mais le service walo | Négatif | Positif | Code-switching darija    |

## 6.3 Sources d'Erreurs Identifiées

En synthèse, les erreurs de classification proviennent de quatre sources principales :

- 1. Avis ultra-courts (45.7% des erreurs) :** La médiane de 2 mots est la cause principale. "bonne expérience", "très utile" en 2-3 mots, le modèle n'a pas assez de contexte et penche vers neutre par défaut.
- 2. Labels incohérents (25% des erreurs) :** L'utilisateur a mis 4-5 étoiles mais le texte décrit un problème (ou inversement). Le modèle classe correctement le ton du texte, mais le label dérivé de la note contredit ce ton.
- 3. Ironie et sarcasme :** "Super application, elle crash toutes les 5 minutes », le modèle identifie "Super" comme marqueur positif sans percevoir l'intention sarcastique.
- 4. Code-switching français/darija :** "L'app mzyana mais le service walo », CamemBERT, entraîné exclusivement sur du français, ne comprend pas "walo" (= rien/mauvais en darija) et ignore cette information critique.

## 7. Déploiement Interactif (Application Gradio)

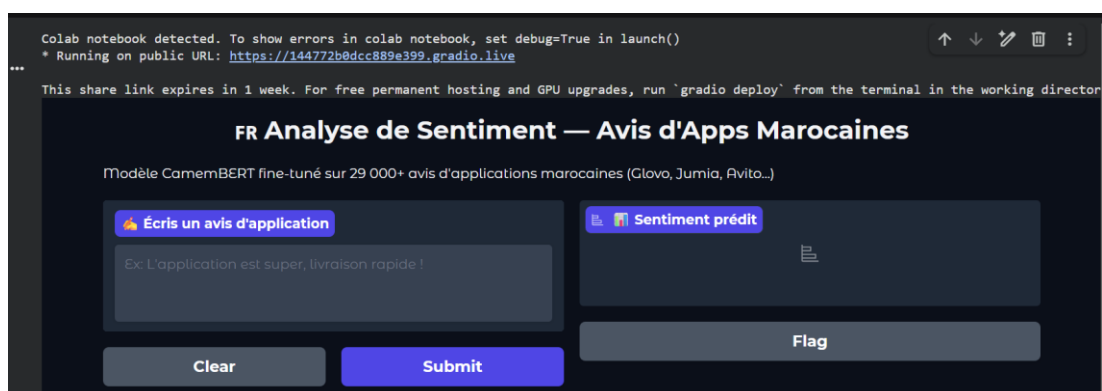
### 7.1 Objectif du Déploiement

Pour démontrer l'applicabilité de notre modèle dans un contexte réel, nous avons développé une interface web interactive à l'aide de la bibliothèque Gradio. Cette application permet à n'importe quel utilisateur de tester notre meilleur modèle (CamemBERT Fine-Tuné) en tapant un avis et en obtenant la prédiction de sentiment en temps réel.

### 7.2 Architecture de l'Application

L'application repose sur les composants suivants :

- Backend (Inférence) : Le modèle CamemBERT fine-tuné et son tokenizer (chargés depuis notre sauvegarde) traitent le texte brut saisi par l'utilisateur.
- Frontend (Interface Web) : Une interface générée automatiquement par Gradio, comprenant une zone de texte en entrée et une zone de résultat en sortie, affichant le sentiment prédit (Positif, Négatif ou Neutre) ainsi que le score de confiance (probabilité) issu de la fonction Softmax.



*Figure 18: interface de Gradio*

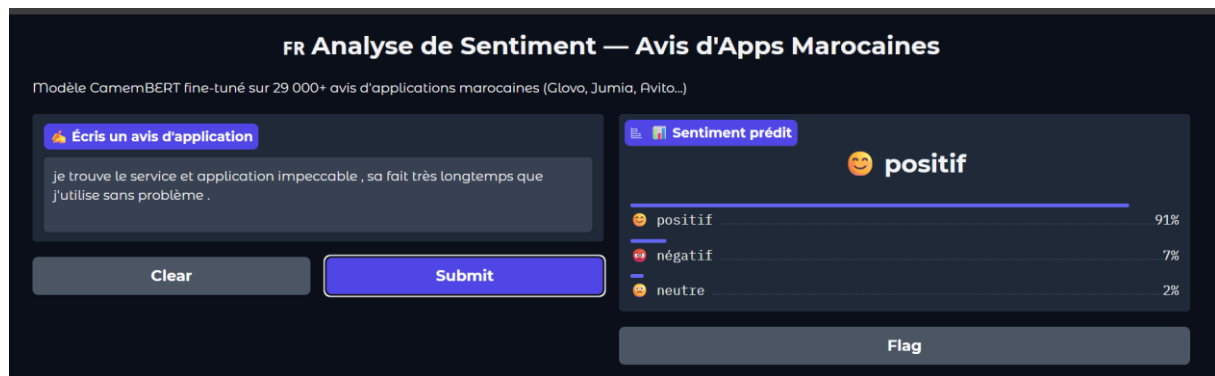


Figure 19: exemple de classification d'avis positif



Figure 20: exemple de classification d'avis negatif



Figure 21: exemple de classification d'avis neutre

## 7.3 Valeur Ajoutée

Cette interface prouve que le modèle dépasse le stade théorique du notebook et peut être facilement encapsulé pour un usage applicatif. Elle permet également aux parties prenantes

de tester le modèle "en direct" et d'expérimenter concrètement les capacités de compréhension sémantique de CamemBERT, ainsi que ses limites sur l'ironie.

## 8. Conclusion et Perspectives

### 8.1 Synthèse des résultats

Ce projet a permis d'implémenter et de comparer quatre approches de classification de sentiments sur un corpus de 29 405 avis d'applications marocaines et africaines. Plusieurs enseignements majeurs en ressortent.

Le F1-macro s'impose comme la métrique de référence pour évaluer un classifieur sur un dataset déséquilibré. L'accuracy, bien que plus intuitive, peut masquer un biais important en faveur de la classe dominante comme l'illustre le cas du TF-IDF + SVM, qui affiche une accuracy de 83 % mais un F1-macro de seulement 57 %.

Les modèles de langue pré-entraînés surpassent nettement les approches classiques, confirmant la supériorité des représentations contextuelles sur les représentations statiques : CamemBERT atteint un F1-macro de 0,63, contre 0,60 au mieux pour les approches One-Hot et TF-IDF.

Le fine-tuning n'apporte qu'un gain marginal par rapport à l'utilisation des embeddings gelés de CamemBERT (+0,0002 en F1-macro), ce qui suggère que les représentations pré-entraînées capturent déjà une information suffisamment riche pour cette tâche.

Enfin, le plafond de performance observé semble davantage imputable à la nature des données, ambiguïté de la classe neutre, présence d'ironie, code-switching français/darija , qu'aux limites intrinsèques des modèles utilisés.

### 8.2 Perspectives

Plusieurs améliorations peuvent être envisagées afin de renforcer les performances et l'efficacité du système. Il serait intéressant d'utiliser des modèles multilingues comme **XLM-RoBERTa** afin de mieux analyser les avis contenant plusieurs langues, notamment le français et l'arabe, qui sont fréquents dans les commentaires marocains. L'enrichissement des données à travers des techniques comme la paraphrase automatique ou la traduction inversée permettrait également d'améliorer la représentation des classes moins fréquentes, en particulier la classe neutre. D'autres approches d'apprentissage pourraient aussi être étudiées pour améliorer la capacité du modèle.



Par ailleurs, le système pourrait évoluer vers une analyse plus détaillée des avis en identifiant les différents aspects mentionnés par les utilisateurs, comme la qualité du service, le prix, la livraison ou l'interface, tout en associant un sentiment spécifique à chaque aspect. Une mise en production via une API REST permettrait d'analyser les nouveaux avis en temps réel et de mettre en place un tableau de bord pour suivre l'évolution des sentiments selon les applications, les catégories et les périodes.

Enfin, l'ajout d'un module de détection du sarcasme permettrait d'améliorer la précision du modèle face aux avis ambigus. L'élargissement du corpus à d'autres pays africains francophones représenterait également une perspective intéressante pour développer un modèle d'analyse de sentiment adapté à un contexte linguistique et culturel plus large.